

1.0 Introdução

Um autômato de pilha é uma máquina abstrata semelhante ao autômato finito, mas com uma pilha como memória auxiliar. Sua função de transição utiliza o estado atual, o símbolo de entrada e o símbolo no topo da pilha para determinar o próximo movimento. Esse modelo pode realizar transições vazias e permite não determinismo, sendo capaz de reconhecer linguagens livres de contexto. sendo mais poderoso que um autômato finito, pois consegue lidar com estruturas que exigem memória, como parênteses balanceados e linguagens do tipo: $L=\{a^n b^n | n \geq 1\}$.

Formalmente, o comportamento de um autômato de pilha é determinado por sua função de transição. Essa função considera três elementos principais: o estado atual, o próximo símbolo da entrada e o símbolo que está no topo da pilha. A partir dessas informações, o autômato decide qual será o próximo estado e qual operação será realizada na pilha, podendo empilhar, desempilhar ou manter símbolos (SIPSER, 2005).

2.0 Desenvolvimento atividade autômato de pilha

No presente trabalho a linguagem utilizada é $L=\{0^n 1^n | n \geq 1\}$, definida sobre o alfabeto $\Sigma=\{0,1\}$. Essa linguagem contém cadeias formadas por uma sequência de zeros seguida pela mesma quantidade de uns. Uma gramática livre de contexto que gera essa linguagem é composta pelas produções:

$$S \rightarrow 0S1;$$

$$S \rightarrow 01.$$

Sendo S o símbolo inicial.

Uma gramática para essa linguagem pode ser: $G=(V,\Sigma,P,S)$ onde:

Elemento	Definição
(V)	({S})
(Σ)	({0,1})
(P)	regras de produção
(S)	símbolo inicial

Considerando a linguagem $L=\{a^n b^n | n \geq 1\}$ e usando o símbolo X para empilhar, a tabela funcionamento do autômato de pilha tendo como exemplo a palavra 000111 é apresentada a seguir.

Símbolo lido	Ação na pilha	Pilha
0	empilha X	X
0	empilha X	X X
0	empilha X	X X X
1	desempilha X	X X
1	desempilha X	X
1	desempilha X	vazia

Conforme a tabela acima, para cada símbolo 0 lido, o autômato empilha um X. Depois, para cada símbolo 1 lido, o autômato desempilha um X. Se ao final da leitura a pilha estiver vazia, a palavra é aceita.

3.0 Algoritmo do código do autômato de pilha na linguagem c++

```
// Autor: Paulo Sergio Theodoro

// Descrição: Autômato de pilha para reconhecer  $L = \{0^n 1^n \mid n \geq 1\}$ 

#include <iostream>
#include <stack>
#include <vector>
#include <string>

using namespace std;

bool automato_pilha(string palavra) {
    stack<char> pilha;
    string estado = "q0";

    for (char simbolo : palavra) {
        if (estado == "q0") {
            if (simbolo == '0') {
                pilha.push('x'); // Empilha x para cada 0 lido
            }

            else if (simbolo == '1') {
                if (pilha.empty()) {
                    return false;
                }

                pilha.pop(); // Desempilha x
                estado = "q1"; // Muda para o estado q1
            }

            else {
                return false; // Rejeita símbolos diferentes de 0 e 1
            }
        }

        else if (estado == "q1") {
            if (simbolo == '1') {
                if (pilha.empty()) {
                    return false;
                }

                pilha.pop(); // Desempilha x para cada 1 lido
            }
        }
    }
}
```

```

        else {
            return false; // Rejeita se aparecer 0 depois de 1
        }
    }
}

if (pilha.empty() && estado == "q1") {
    return true;
} else {
    return false;
}
}

int main() {

    // Testes
    vector<string> palavras = {
        "01",
        "0011",
        "011",
        "001",
        "000111",
        "0111",
        "00011",
        "a1",
        "(1",
        "0)",
        "00001111"
    };

    for (string palavra : palavras) {
        if (automato_pilha(palavra)) {
            cout << palavra << " => aceita" << endl;
        } else {
            cout << palavra << " => rejeitada" << endl;
        }
    }

    return 0;
}

```

Resultados para as palavras testes:

```

01 => aceita
0011 => aceita
011 => rejeitada
001 => rejeitada
000111 => aceita

```

0111 => rejeitada 00011 => rejeitada a1 => rejeitada (1 => rejeitada 0) => rejeitada 00001111 => aceita
--

4.0 Referência Bibliográficas

SIPSER, Michael. *Uma introdução à teoria da computação*. Tradução de Ruy J. Guerra B. de Queiroz. Versão parcial de 11 de fevereiro de 2005. Tradução de: *Introduction to the Theory of Computation*. PWS Publishing Company, 1997.